

# Streams, Friends & the Many Ways to Build an Object

Quiz 03 — ANSWER KEY

## array[0] Find the Bug

model answer

Bug is on `x = x; y = y;` inside the constructor body: the parameters `x` and `y` are assigned to *themselves* (the parameter names hide the member names), so the data members `Point::x` and `Point::y` are never touched and stay whatever garbage was on the heap/stack — hence `(0, 0)` or worse.

### CORRECTED CONSTRUCTOR

```
Point(int x, int y) : x(x), y(y) {}
```

In the initializer list, `x(x)` is unambiguous: the left `x` is looked up as a member, the right `x` as the parameter in scope — unlike a bare assignment in the body, where an unqualified `x` always resolves to the parameter.

## array[1] Write the Code

model answer

### (A) INSIDE THE CLASS

```
friend ostream& operator<<(ostream& os, const Fraction& f);
```

### (B) FREE FUNCTION, BELOW THE CLASS

```
ostream& operator<<(ostream& os, const Fraction& f) {
    return os << f.num << "/" << f.den;
}
```

### GRADING NOTES

#### What counts

##### array[0] — bug

Must name the exact issue (self-assignment / parameter shadows member), not just "wrong values." Line number or the literal `x = x;` both acceptable.

##### array[0] — fix

`: x(x), y(y)` in either member order is fine here (no dependency between them). Renaming the parameters (e.g. `_x`, `xVal`) and assigning in the body also earns full credit — the point is members get set.

##### array[1](a) — watch

Accept the friend line placed anywhere inside the class body. A student who instead makes `num/den public`: has NOT answered the question — that avoids `friend` rather than using it.

##### array[1](b) — watch

Must return `ostream&` (not `void`) and take `ostream&` first, `const Fraction&` second. A member function version (`Fraction::operator<<`) is incorrect for this call direction — partial credit if the friend declaration in (a) was also dropped consistently.

##### Cosmetic

Separator character (`/`) must appear; exact spacing does not matter.